

University of Chicago - Spring 2026

(Honors) complexity theory

Vincent X. Wang

October 6, 2026

These are notes based on lectures in CMSC 28130 / MATH 28330 at University of Chicago, taught by Janos Simon during Autumn 2026.

Contents

<i>Turing machines</i>	2
<i>Introduction (9/28/26)</i>	2
<i>Universal Turing machine (9/30/26)</i>	2
<i>Recognizable and decidable (10/2/26)</i>	4
<i>Verification and non-deterministic Turing machines (10/5/25)</i>	5

Turing machines

Introduction (9/28/26)

We set some notation. An **alphabet** Σ is a finite set, with each element a **character**. A **language** is a collection of strings (composed of characters).

We can formulate the goal of computation as determining if a given string is inside the language or not, as every YES/NO query can be encoded in this way. Our goal for the time being is to study the most “braindead” form of computation:

Definition 1. A **Turing machine** is composed of an:

- states Q ;
- alphabet Σ ;
- a one-way infinite tape where each space can be assigned an element of the alphabet (or BLANK)¹;
- a transition function $\delta : Q \times \Sigma \rightarrow Q \times \Sigma \times \{L, R, S\}$;²

Example 2. Let $L = \{xay : x, y \in \{0, 1\}^*, a \in \Sigma\}$. How might we construct δ ? What about $L = \{xaya : x, y \in \{0, 1\}^*, a \in \Sigma\}$?

Assuming Σ is finite, we can easily construct Turing machines *with a finite number of memory states* to determine L .

Example 3. What about $L = \{xax^R\}$? Here, x^R denotes string reversal. In this example, we use the *tape* itself to store information, instead of states.

One may observe that it is much easier to do the above problem on a Turing machine with *two* tapes. A more formal treatment is given in [Sip12], but the overall claim is that without loss of generality, we do not gain any computational power with having two tapes (i.e., anything implementable with two tapes is also implementable with one tape).

Let us provide a sketch, and suppose we are implementing a Turing machine with two tapes. We can simulate this with a Turing machine on one tape. On our Turing machine with one tape, we can use a delimiter $\#$ to create space for and “separate” the two tapes. The trick in implementing this is to define *new* characters that track where the position of the tracker on each of the two tapes is. All of the procedures should be pretty straightforward to simulate with the exception of writing characters onto a BLANK on a tape. This may require us to shift parts of our one-tape Turing machine one character to the right.

Universal Turing machine (9/30/26)

The Church–Turing thesis refers to the claim that an intuitive notion of ‘computability’ exactly one that is computable by a Turing machine. More precisely, we have the following notions:

¹ We can formalize this with more details... in general, the formal details of a Turing machine don’t really matter. (e.g., perhaps we are sloppy because we cannot move left in the tape at the beginning)

² We think of this function δ as “doing computation” on the tape – e.g., an output (p, c, R) may specify a next state, tell us to write down c , and move right.

Definition 4. We say a language L is **accepted** (or recognized) by a Turing machine M if

$$x \in L \iff M(x) \text{ halts and accepts.}$$

Definition 5. We say a language L is **decided** by a Turing machine M if M halts and

$$M(x) = \begin{cases} 1 & x \in L \\ 0 & x \notin L \end{cases}.$$

We also call a recognizable language *computably or recursively enumerable*, and a decidable language *computable or recursive*. Why are recognizable languages called *enumerable*? Consider an “intelligent” printer E that just spews out

$$L_E = \{y_1, y_2, \dots\}.$$

We can construct a Turing machine that just looks at E to recognize L_E . Somewhat conversely, we can construct E via a Turing machine that recognizes L_E by enumerating all the strings and running them simultaneously.

The **universal Turing machine** is a Turing machine that can simulate any other Turing machine. More specifically, U satisfies³

$$U(M, x) = M(x).$$

[Details on implementing such a Turing machine were sketched.] For the rest of the class, **without loss of generality, yes means accepting, and no means to run forever.**

Must every language be decidable? The answer is no.

Theorem 6. *There exists a language L that is not decidable by a Turing machine.*

Proof. A “cheap” way to prove this is via cardinality. Turing machines can be enumerated and thus the decidable languages are countable. But, in general, subsets of strings are not countable. $\square$

A more illuminating example is the following: let

$$\text{HALT}(M, x) = \begin{cases} 1 & \text{if } M \text{ halts with input } x \\ 0 & \text{if } M \text{ does not halt with input } x \end{cases}.$$

Theorem 7. *HALT is not decidable.*

Proof. Suppose there exists a Turing machine DECIDE that decides HALT. Define

$$\text{CONFUSE}(M) := \begin{cases} \text{if } \text{DECIDE}(M, M) = 1, \text{ then go into an infinite loop} \\ \text{if } \text{DECIDE}(M, M) = 0, \text{ halt and accept} \end{cases}.$$

What is DECIDE(CONFUSE, CONFUSE)? In either way, we run into a contradiction. $\square$

³ We tend to use M pretty loosely – here, it is technically a string embedding of M in a scheme we do not really specify. More commonly this might be denoted $\langle M \rangle$.

Recognizable and decidable (10/2/26)

Let

$$A_{\text{TM}} = \{(M, x) : M \text{ accepts } x\}$$

Theorem 8. A_{TM} is computably enumerable, though not decidable.

Proof. We have seen this in the universal Turing machine. If M accepts x , then the universal Turing machine simulates M on x and accepts (M, x) . We can show undecidability via reduction from HALT. $\square$

Consider the language:

$$L_{\text{nonempty}} = \{\langle M \rangle : \exists x \text{ such that } M \text{ accepts } x\}.$$

Theorem 9. L_{nonempty} is computably enumerable, but not decidable.

Proof. Enumerate all strings and *simultaneously* run the universal Turing machine on each (M, x) (though we need to do a silly thing and do some scheme where we run each string for $1, 2, 3, \dots$ operation so that we don't run into any halting issues).

To show that L_{nonempty} is not decidable, we reduce from A_{TM} . Given M, x , construct a machine M' that, if M accepts x , then M' accepts everything (and conversely, if M does not terminate on x , then M' likewise does not terminate). Then, $M' \in L_{\text{nonempty}}$ if and only if $(M, x) \in A_{\text{TM}}$. $\square$

Can we show that there are languages that are not recognizable? The answer is yes! Let $\bar{L} = \text{co}L$ denote the complement of L (i.e., $\{x : x \notin L\}$).

Theorem 10. If L and $\text{co}L$ are recognizable, then L is decidable.

Proof. Run the recognition procedures for L and $\text{co}L$ simultaneously. $\square$

Corollary 11. $\text{co-}A_{\text{TM}}$ is not recognizable.

Here is another undecidable problem:

Theorem 12.

$$EQ = \{\langle M_1, M_2 \rangle : M_1 \text{ equivalent to } M_2\}.$$

is undecidable.

Proof. Suppose EQ is decidable, and fix some (M, x) . Let M_1 be such that on all inputs y , M_1 simulates M on x . Let M_2 accept every input. Then, this would imply that A_{TM} is decidable, as determining if $\langle M_1, M_2 \rangle \in EQ$ is equivalent to determining if $(M, x) \in A_{\text{TM}}$. $\square$

More formally, the tricks here are a *mapping* reduction, where we construct an *computable* function f between the enumerable strings that preserves $y \in L_1 \iff f(y) \in L_2$. We denote this with $L_2 \geq_m L_1$.

Verification and non-deterministic Turing machines (10/5/25)

How can we verify a computation of a Turing machine M on x ?

We sketch a “verifier” Turing machine with state z (that encodes the computation of a M) composed of the tape of M , the state q , and a configuration. Let the configuration at time i be

$$\#y_1 \dots y_m q y_{m+1} \dots y_l \#.$$

Let C_i be the initial config, and let the final state C_{accept} be an accepting state. For all i , verify that C_{i+1} follows from C_i (which can be done in polynomial time).

Theorem 13. *The statement L is recognizable by a Turing machine M is equivalent to the statement that there exists z_x and (computable) $P_M(u, v)$ such that $x \in L \Leftrightarrow \exists z_x P_M(x, z)$, where $P_M(x, z)$ is the predicate that is a Boolean function that verifies that z is a record of x .*

discussion on how this is different for *decidable* rather than recognizable.

In a deterministic Turing machine, we have a transition function (e.g., $\delta : (q, a) \mapsto (p, b, L)$). In a non-deterministic Turing machine, δ is a relation (e.g., $(q, a) \mapsto \{(p, b, L), (r, c, S), \dots\}$). Without loss of generality, δ can map to a set of size 2 [can represent any finite tree with a binary tree trick].

We say a non-deterministic Turing machine N accepts x if there exists a valid computation ending with an accepting state.

Theorem 14. *If L is recognizable by a non-deterministic Turing machine, then L is recognizable by a Turing machine.*

Proof. The “stupid” way is just to generate all strings and check string-by-string as before (note that accepting for a non-deterministic Turing machine is just an *existence* statement, not that we always get to an accepting state). The second way is to think of the computation as a “tree” and just search the tree. $\square$

Example 15 (Logical proof systems). As an example, we think of high-school geometry. A logical proof system is equipped with *axioms* and *rules of inference*. These build up to *theorems*. Then, we can conceive a non-deterministic Turing machine that encodes the rules of inference that comes up with all “theorems.” Thus, a logical proof system is equivalent to a computationally enumerable language.

A logical proof system is *decidable* if it is equivalent to a decidable language (i.e., there exists a Turing machine that halts).

A *linear bounded automata* is a Turing machine with a fixed-length tape at input, but can have a bigger alphabet.

Theorem 16. A_{LBA} is decidable, as

$$A_{LBA} = \{x : x \in L(M)\}.$$

References

- [Sip12] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3 edition, 2012.